Scheduling examples

Starting Balance

A	B
$50	$200

Serial schedule ***T_2,T_1:***

T_1: A += 100 | B -= 100

T_2: A *= 1.06 | B *= 1.06

A	B
$153	$112

Interleaved schedule B:

T_1: A += 100 | B -= 100

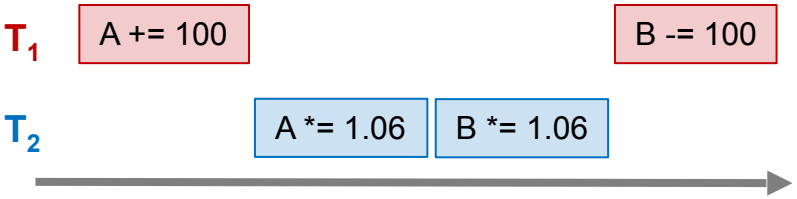
T_2: A *= 1.06 | B *= 1.06

A	B
$159	$112

Different result than serial T_2,T_1 ALSO!

Scheduling examples

Interleaved schedule B:



This schedule is different than ***any serial order!*** We say that it is **not serializable**

Scheduling Definitions

- A **<u>serial schedule</u>** is one that does not interleave the actions of different transactions
- A and B are **<u>equivalent schedules</u>** if, ***for any database state***, the effect on DB of executing A **is identical to** the effect of executing B
- *A* **<u>serializable schedule</u>** is a schedule that is equivalent to ***some*** serial execution of the transactions.

The word "**some"** makes this definition powerful & tricky!

Serial Schedules

T1		A += 100	B -= 100		
T2				A *= 1.06	B*= 1.06

S1

T1				A += 100	B -= 100
T2		A *= 1.06	B *= 1.06		

S2

Interleaved Schedules

T1		A += 100		B -= 100	
T2			A *= 1.06		B*= 1.06

S3

T1			A += 100		B -= 100
T2		A *= 1.06		B *= 1.06	

S4

T1			A += 100	B -= 100	
T2		A *= 1.06			B*= 1.06

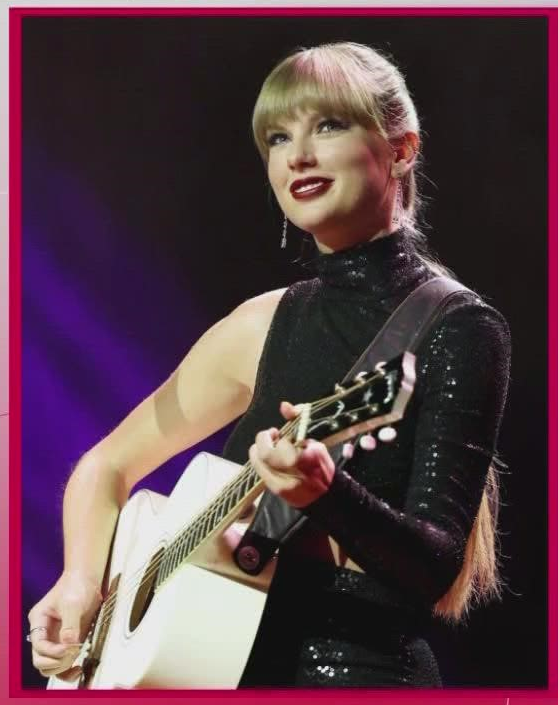
S5

T1		A += 100			B -= 100
T2			A *= 1.06	B *= 1.06	

S6

Serial Schedules	S1, S2
Serializable Schedules	S3, S4 (And S1, S2)
Equivalent Schedules	<S1, S3> <S2, S4>
Non-serializable (Bad) Schedules	S5, S6

instagram.com/taylorswift



Well. It goes without saying that I'm extremely protective of my fans. We've been doing this for decades together and over the years, I've

"...excruciating for me to just watch mistakes happen with no recourse."

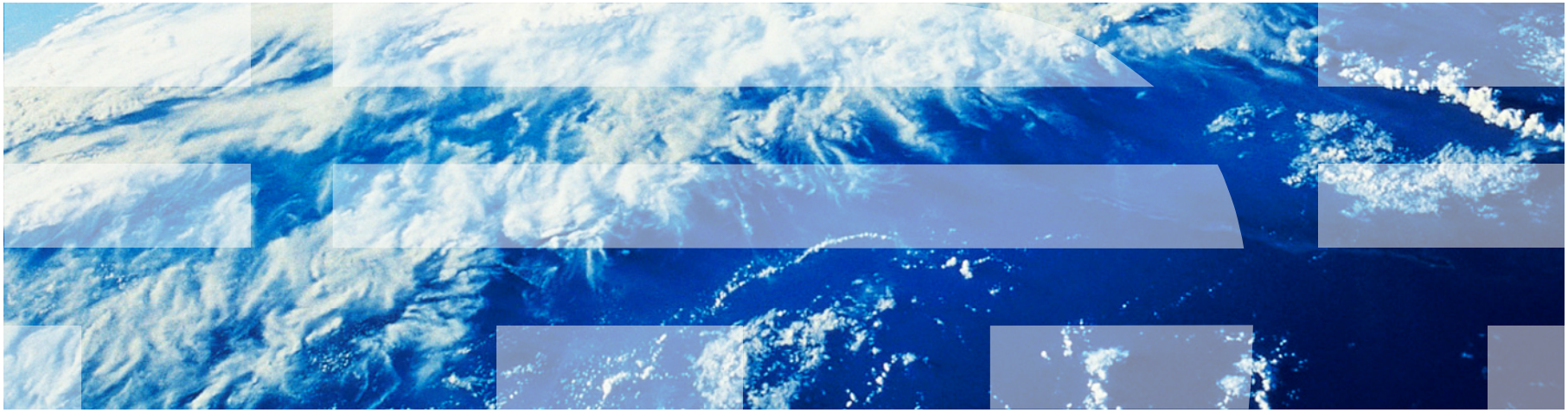
There are a multitude of reasons why people had such a hard time trying to get tickets and I'm trying to figure out how this situation can be

TAYLOR SWIFT SPEAKS OUT ON TICKET ISSUES

10TAMPABAY.COM 5:42 66°

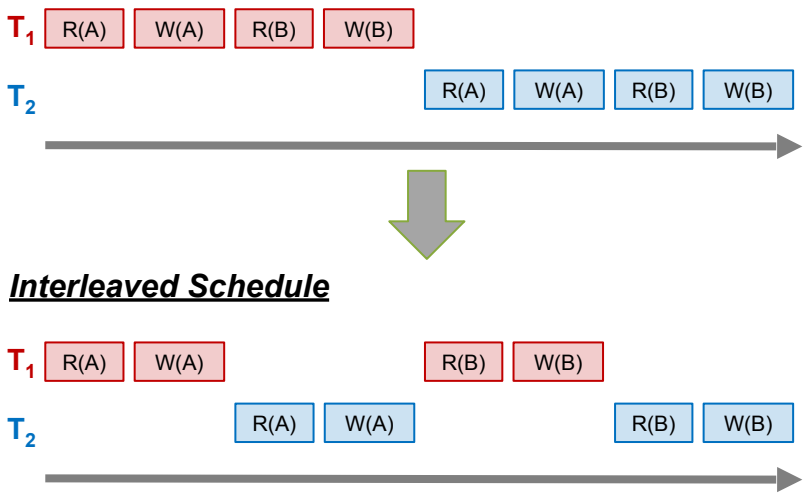
https://www.educative.io/blog/taylor-swift-ticketmaster-meltdown

Conflicts and Anomalies



General DB model:
Concurrency as Interleaving TXNs

Serial Schedule



Each action in the TXNs *reads a value* and then *writes one back*

For our purposes, having TXNs occur concurrently means **interleaving their component actions (R/W)**

We call the particular order of interleaving a **schedule**

Conflict Types

Two actions **conflict** if they are part of different TXNs, involve the same variable, and at least one of them is a write

Thus, there are three types of conflicts:

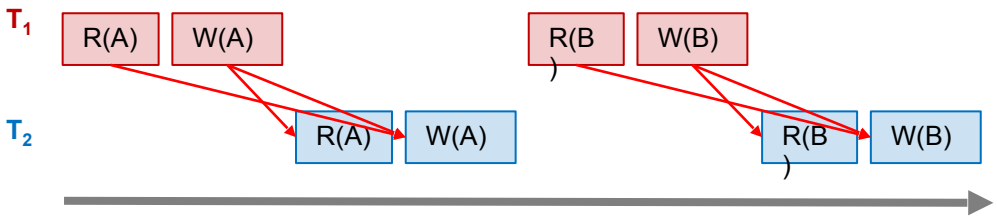
- Read-Write conflicts (RW)
- Write-Read conflicts (WR)
- Write-Write conflicts (WW)

Why no "RR Conflict"?

Note: **conflicts** happen often in many real world transactions. (E.g., two people trying to book an airline ticket)

Conflicts

Two actions **conflict** if they are part of different TXNs, involve the same variable, and at least one of them is a write



All “conflicts”!

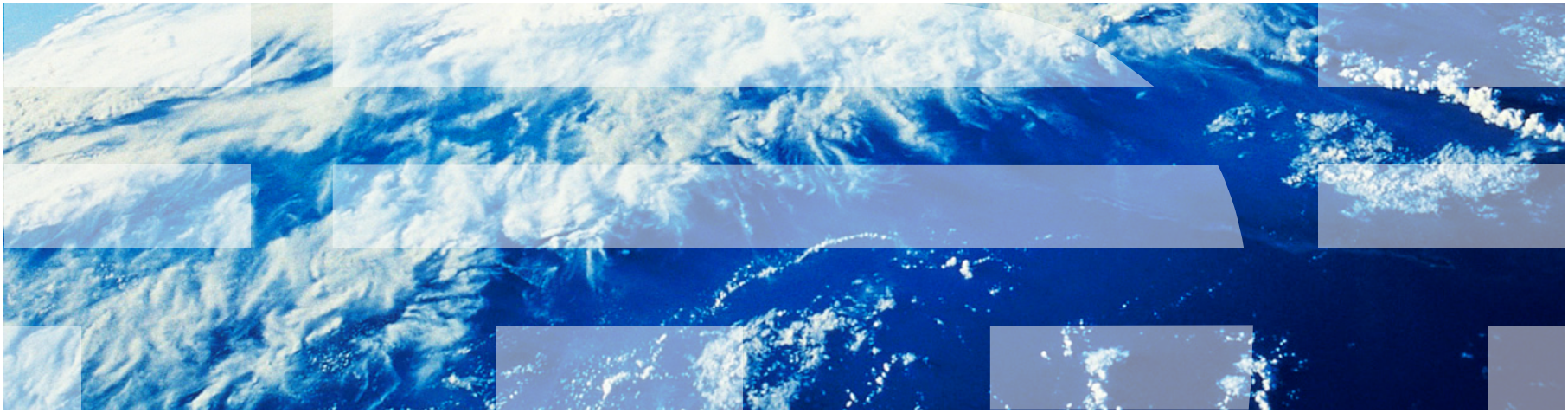
Note: Conflicts vs. Anomalies

Conflicts are in both “good” and “bad” schedules
(they are a property of transactions)

Goal: Avoid Anomalies while interleaving transactions with conflicts!

- Do not create “bad” schedules where isolation and/or consistency is broken (i.e., Anomalies)

Conflict Serializability



Conflict Serializability

Two schedules are **conflict equivalent** if:

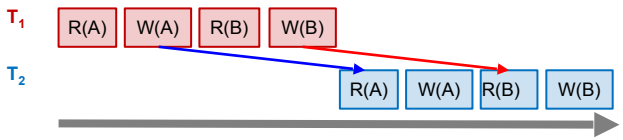
- *Each TXN's order of operations is the same*
- Every *pair of conflicting actions* of two TXNs are *ordered in the same way*

Schedule S is **conflict serializable** if S is *conflict equivalent* to some serial schedule

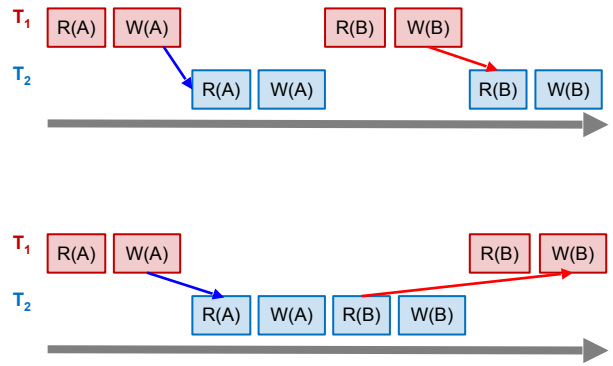
Conflict serializable ⇒ serializable
So if we have conflict serializable, we have consistency & isolation!

Example **“Good” vs. “bad” schedules**

Serial Schedule:



Interleaved Schedules:

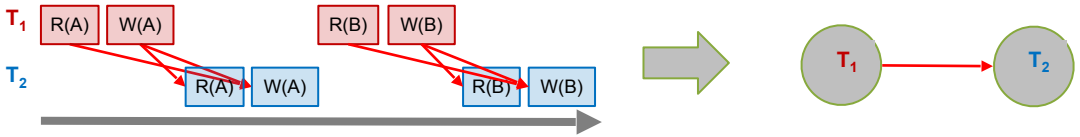


Note that in the “bad” schedule, the ***order of conflicting actions is different than the above (or any) serial schedule!***

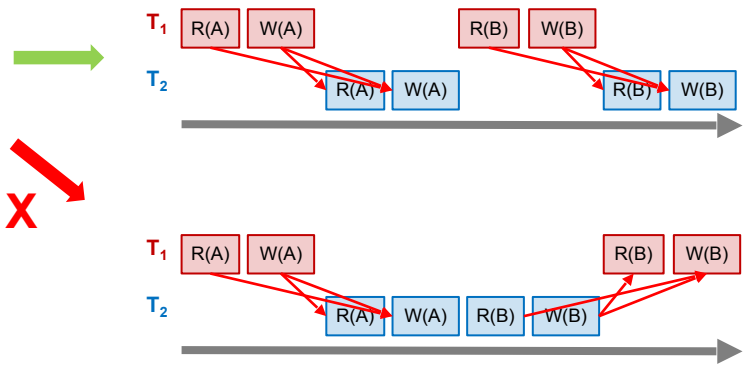
Conflict serializability provides us with an operative notion of “good” vs. “bad” schedules! “Bad” schedules create data Anomalies

The Conflict Graph

- Let's now consider looking at conflicts **at the TXN level**
- Consider a graph where the **nodes are TXNs**, and there is an edge from $T_i \rightarrow T_j$ **if any actions in T_i <u>precede and conflict with</u> any actions in T_j**



What can we say about “good” vs. “bad” conflict graphs?

Serial Schedule:

Interleaved Schedules: